

# Technical Design Document

## Real-Time Speech Translation System

Manmadha (2025MCS2101)  
Jithendra (2025MCS2743)

February 27, 2026

### 1 Overview / Problem Statement

The objective of this project is to design and implement a real-time speech-to-speech translation system for Indian languages. The system must process live microphone input and produce translated speech output with bounded latency while operating continuously in a streaming manner.

Unlike traditional batch-based systems that process complete audio files, this system must operate incrementally. Audio must be captured, processed, and forwarded through the pipeline in small frames, ensuring responsiveness and low delay.

The long-term system will integrate Automatic Speech Recognition (ASR), machine translation, and Text-to-Speech (TTS) modules. The current implementation establishes the streaming backbone and observability infrastructure necessary for incremental integration.

### 2 Goals

- Process live microphone input in real time.
- Maintain low latency through streaming-based processing.
- Provide a real-time monitoring dashboard.
- Design a modular, extensible pipeline.
- Ensure fully offline execution.

### 3 System Architecture

#### 3.1 Target Final Architecture

The complete system will follow a pipelined streaming architecture:

Microphone  $\rightarrow$  ASR  $\rightarrow$  Translation  $\rightarrow$  TTS  $\rightarrow$  Speaker

Each stage operates independently and communicates via streaming interfaces.

## 3.2 Phase 1 Architecture

The current implementation establishes the foundational skeleton:

Microphone → Audio Buffer → Monitoring Stage → Web Dashboard

This stage ensures:

- Continuous audio capture
- Decoupled producer–consumer architecture
- Independent processing stages
- Real-time observability

## 3.3 Concurrency Model

The system employs a producer–consumer model:

- The microphone callback acts as a producer.
- Audio frames are inserted into a bounded queue.
- A monitoring thread consumes frames independently.
- FastAPI runs an asynchronous WebSocket server.
- The browser receives live updates at approximately 10 Hz.

This separation ensures that audio capture is never blocked by downstream processing.

# 4 Data Model and Interfaces

## 4.1 Audio Frame Representation

Audio is processed in fixed-size frames:

- Sampling Rate: 16 kHz
- Frame Duration: 20 ms
- Samples per Frame: 320
- Data Type: Floating-point numpy array

Frame-level processing allows incremental streaming and low latency.

## 4.2 Shared System State

The system maintains a shared state object containing:

- Current audio energy value
- Total frames processed
- System start time
- Liveness indicator

This state is continuously updated by the monitoring thread and streamed to the dashboard.

## 4.3 WebSocket Interface

The dashboard connects via:

`/ws`

The server sends JSON objects with the following structure:

```
{  
  "energy": float,  
  "frames": int,  
  "runtime": float,  
  "alive": bool  
}
```

This enables real-time visualization without polling.

## 5 Design Decisions and Alternatives Considered

### 5.1 WebSockets vs HTTP Polling

WebSockets were chosen to enable bidirectional, low-latency communication. HTTP polling was rejected due to higher overhead and reduced responsiveness.

### 5.2 Bounded Queue vs Unbounded Buffer

A bounded queue prevents uncontrolled memory growth and enforces backpressure. An unbounded buffer could lead to instability under sustained load.

### 5.3 Multithreading vs Single Thread

A single-threaded architecture would cause blocking between audio capture and processing. Multithreading ensures real-time responsiveness.

## 5.4 Offline Execution vs Cloud Services

Cloud-based APIs were not selected because they introduce network latency and violate the fully offline requirement.

## 6 Milestones and Timeline

The project is divided into clearly defined phases:

---

|         |                                                   |
|---------|---------------------------------------------------|
| Phase 1 | Audio Ingress and Dashboard (Completed)           |
| Phase 2 | Streaming Speech Recognition Integration          |
| Phase 3 | Incremental Translation Module                    |
| Phase 4 | Real-Time Speech Synthesis                        |
| Phase 5 | End-to-End Latency Optimization and Demonstration |

---

Each phase incrementally integrates one major component into the streaming backbone.

## 7 Performance Considerations

To ensure real-time performance:

- Frame size is limited to 20 ms to reduce buffering delay.
- Queue size is bounded to avoid memory overflow.
- Processing is decoupled to avoid blocking.
- Dashboard updates occur at controlled frequency.

The final system targets:

- End-to-End Latency  $< 2$  seconds
- Real-Time Factor (RTF)  $< 1$
- Stable continuous runtime

## 8 Repository

The project source code is hosted at:

`github.com/ManmadhaKuttum/CornerStone_COP7001`

Access has been granted to the evaluator.

## 9 Platform Target

The current implementation targets:

- P1 – x86 Laptop/Desktop (Baseline Platform)

Future work may explore deployment on embedded platforms.